Energy-Aware Scheduling in big.LITTLE Architectures

Comparative Analysis and Bonus Scheduler Design

Submitted by:

Shubham Balaso Jadhav
[CS24S009]
[CS6200: Advanced Computer Architecture]

IIT Madras

Date: 28/04/2025

Abstract

This report presents an analysis and implementation of scheduling strategies for a big.LITTLE heterogeneous processor system, focusing on minimizing the Energy Delay Product (EDP). Three standard schedulers - Clustering, Switcher, and Global - were first developed based on fixed policies. Building upon their strengths and weaknesses, a novel Bonus Scheduler was designed to adaptively classify processes as CPU-bound, IO-bound, or Mixed, and dynamically assign them to appropriate cores while optimizing core power states. Experimental results across four diverse workloads show that the Bonus Scheduler consistently achieves superior EDP compared to traditional schemes, offering a balanced trade-off between energy efficiency and performance. This study highlights the importance of workload-adaptive, energy-aware scheduling policies in modern heterogeneous computing platforms.

Contents

0.1 Introduction . . . 2
0.1.1 Background on big.LITTLE Architecture . . . 2
0.1.2 Importance of Energy Delay Product (EDP) in Scheduling . . . 2
0.2 System and Simulator Overview . . . 2
0.2.1 Simulator Assumptions . . . 2
0.2.2 Core and Process Modeling . . . 3
0.3 Scheduler Implementations . . . 3
0.3.1 Clustering Scheduler . . . 3
0.3.2 Switcher Scheduler . . . 4
0.3.3 Global Task Scheduler . . . 4
0.3.4 Strengths and Limitations Comparison . . . 5
0.3.5 Schedule and Types of workload for better performance . . . 5
0.4 Bonus Scheduler . . . 6
0.4.1 Objective . . . 6
0.4.2 Key Design Principles . . . 6
0.4.3 Scheduler Features . . . 6
0.4.4 Achievements . . . 8
0.4.5 Summary of Improvements over Base Schedulers . . . 9
0.4.6 Conclusion . . . 9
0.4.7 Bonus Scheduler Decision Flow Diagram . . . 10
0.5 Performance Evaluation . . . 11
0.5.1 Test Workloads Description . . . 11
0.5.2 Energy Delay Product (EDP) Comparison . . . 11
0.5.3 Graphical Results . . . 11
0.6 Extra Bonus Scheduler Strategy . . . 12
0.6.1 Suitability-Guided/Distance-Based Scheduling: Implementation Strategy 12
0.6.2 Energy Prediction-Based Scheduling: Implementation Strategy . . . 13
0.7 Conclusion . . . 14
0.8 References . . . 14

0.1 Introduction

0.1.1 Background on big.LITTLE Architecture

big.LITTLE is a heterogeneous multi-processing architecture developed by ARM Holdings, where two types of cores are integrated within a single processor:

- **Big cores**: High-performance but power-hungry cores.
- **LITTLE cores**: Energy-efficient but lower-performance cores.

The idea behind big.LITTLE is to dynamically switch between cores based on the computational requirements of running processes. High-load, compute-intensive tasks can be assigned to Big cores for maximum performance, while lighter tasks can run on LITTLE cores to conserve energy. This architecture is widely used in mobile System-on-Chips (SoCs), balancing performance with battery life.

0.1.2 Importance of Energy Delay Product (EDP) in Scheduling

Energy Delay Product (EDP) is a metric that captures the trade-off between energy consumption and performance (execution time). It is calculated as:

$$EDP = EnergyConsumed \times TurnaroundTime$$

Lower EDP values represent better overall system efficiency. In the context of big.LITTLE scheduling, minimizing EDP is crucial as it ensures processes are completed quickly without excessive energy consumption. Effective scheduling strategies must, therefore, intelligently assign tasks to Big or LITTLE cores based on past process behavior, achieving an optimal balance between speed and energy use.

0.2 System and Simulator Overview

0.2.1 Simulator Assumptions

To simplify the complexity of heterogeneous multi-core scheduling, the simulator used in this study incorporates the following assumptions:

- Each process is modeled as a sequence of CPU and IO bursts.
- CPU bursts vary in execution time depending on the core (Big or LITTLE), but IO bursts are independent of core type.
- Core properties are fixed: No hyper-threading, no dynamic frequency scaling.
- Cores can be powered ON/OFF instantaneously with no delay.
- Processes are non-preemptive once scheduled on a core, except during explicit migration.

- All cores have defined MIPS (Millions of Instructions per Second) rates, idle power, and running power values.
- Scheduling decisions are made only at epoch boundaries (every 100 ms).
- Turnaround time and energy consumption are the key metrics for evaluation, combined via EDP.

0.2.2 Core and Process Modeling

Core Modeling:

- Each core maintains its own state: powered ON, powered OFF, idle, or running a process.
- Each core records its load and energy consumption history.
- The speed (MIPS) and power (idle and running) are fixed per core.

Process Modeling:

- Processes transition through states: NOTARRIVED, ARRIVED, READY, RUNNING, WAITING_FOR_IO, TERMINATED.
- CPU bursts execute instructions based on core MIPS rate; IO bursts simply wait for a given time.
- Migration penalties are modeled by pausing execution when a process switches cores.
- Completed CPU and IO bursts are recorded to allow adaptive scheduling based on process history.

0.3 Scheduler Implementations

0.3.1 Clustering Scheduler

Idea:

- One cluster active at a time: Either all Little cores (0–3) or all Big cores (4–7) are powered ON.
- Load-based switching:
  - If Little cluster's average load $\geq 50\%$, migrate to Big cluster.
  - If Big cluster's average load $< 50\%$, migrate back to Little cluster.
- New Process Assignment: Processes are assigned Round-Robin among currently active 4 cores.

Implementation:

- Tracks which cluster (Little or Big) is active.

- Migrates processes at epoch boundary based on cluster load.
- Powers ON/OFF entire clusters during switching.
- Uses 4-core Round Robin to assign arriving processes (RR pointer rotates over 4 active cores).

0.3.2 Switcher Scheduler

Idea:

- Paired Core Activation: Each Little core is paired with a Big core (0↔4, 1↔5, etc.).
- Per-core switching instead of cluster-wide switching:
  - If Little core's load $\geq 50\%$, migrate to corresponding Big core.
  - If Big core's load $< 50\%$, migrate back to Little core.

Implementation:

- Checks load for each active core independently.
- Migrates processes only between paired cores.
- Processes assigned in Round-Robin across active cores.

0.3.3 Global Task Scheduler

Idea:

- All 8 cores ON permanently.
- No migrations at all.
- Simply assign processes Round-Robin across all 8 cores.

Implementation:

- Every process is assigned in order to the next available core.
- No dynamic behavior (no switching, no migrations).

0.3.4 Strengths and Limitations Comparison

Scheduler	Strengths	Limitations
Clustering Scheduler	• Energy efficient for light workloads (mostly Little cores active). • Simple and intuitive switching strategy.	• Switching entire cluster is heavy; all processes are migrated at once. • Big performance penalty during migrations (migration penalty time).
Switcher Scheduler	• More fine-grained control compared to Clustering. • Energy saving: Only migrate processes that need it. • Reduces massive migrations (localized switching).	• Some unnecessary migrations if process load fluctuates slightly around 50%. • Does not differentiate between CPU-bound and IO-bound processes.
Global Task Scheduler	• Best performance in CPU-bound scenarios (all cores active). • Simple, no migration overhead.	• Energy inefficient: even idle cores stay powered ON. • Poor choice for IO-bound or lightly loaded systems.

Table 1: Comparison of Clustering, Switcher, and Global Task Schedulers

0.3.5 Schedule and Types of workload for better performance

Scheduler	Best Workloads	Reason
Clustering	Workload 1, 2	Works best when process behavior is static and classification is reliable (e.g., fully IO-bound or fully CPU-bound workloads). Low migration overhead.
Switcher	Workload 3	Adaptively reclassifies processes and migrates between clusters. Excels in mixed or dynamically shifting workloads.
Global	–	Flexible but unoptimized. No core affinity leads to poor energy efficiency. Performs suboptimally in all workloads.
Bonus (Custom)	Workload 1, 2, 3, 4	Dynamically adjusts thresholds, aggressively migrates, and packs tasks for optimal EDP. Outperforms others across all workload types.

Table 2: Best Performing Workloads by Scheduler

0.4 Bonus Scheduler

0.4.1 Objective

Design an energy-aware scheduler for a heterogeneous big.LITTLE architecture that optimizes Energy Delay Product (EDP) across a wide variety of workloads while strictly adhering to constraints on information usage.

0.4.2 Key Design Principles

Only Past Information:

- Classification and scheduling decisions rely strictly on past CPU/IO bursts already completed.
- No future burst lengths or pending burst details are accessed.

Dynamic Process Classification:

- Processes are dynamically classified into:
  - **CPU-bound**: Long average CPU bursts, low IO ratio.
  - **IO-bound**: High ratio of IO bursts.
  - **Mixed**: Intermediate behavior.
- Classification is updated after each epoch based on completed burst history.

Optimistic First Placement:

- New processes with no past history are initially assumed to be CPU-bound and are scheduled onto Big cores.
- Migration corrects early misplacements as more information becomes available.

0.4.3 Scheduler Features

1. Idle Core Power Optimization (Core Packing)

After migrating IO-bound processes to Little cores, we consolidate them onto as few cores as possible and power off the remaining idle cores—even among the Little cores.

Rationale:

- Idle cores still consume idle power (e.g., Little cores consume 1W when idle).
- Fewer powered cores result in less static energy waste.

Implementation:

- When scheduling or migrating IO-bound processes, always choose the least-loaded Little core.
- Power off any cores that have no processes scheduled.

Principle: Consolidate processes into the fewest active cores possible.

2. Prefer Busy Big Cores for CPU-bound Processes

CPU-bound processes are scheduled on Big cores, but we prefer cores that are already powered on and running other processes.

Rationale:

- Activating a powered-off core introduces energy overhead.
- Keeping an active core busy is more efficient.

Implementation:

- Prefer Big cores that are already on and have fewer than 2–3 processes.
- Power on a new Big core only if necessary.

3. Smarter Initial Classification (Early Bias)

Instead of always assuming new processes are CPU-bound, we bias initial classification based on process arrival patterns.

Observation:

- Clustered arrivals typically indicate IO-heavy workloads (e.g., bursty IO apps).
- Sparse arrivals suggest CPU-heavy workloads.

Heuristic:

- Many arrivals at the same epoch ⇒ classify as MIXED or IO-bound.
- Spaced-out arrivals ⇒ classify as CPU-bound.

4. Dynamic Threshold Adjustment

Thresholds for process classification are adjusted dynamically based on system load.

[language=C++, caption=Adaptive thresholds in code] const double $\text{CPU}_B OUND_T HRESHOLD = 100; //msconstdouble IO_B OUND_T HRESHOLD = 0.6;$

Logic:

- If system load $> 70\%$ ⇒ favor energy savings (Little cores).
- If system load $< 40\%$ ⇒ favor performance (Big cores).

5. Predictive Pre-Power Management

Rather than powering off cores immediately after becoming idle, we anticipate future arrivals.

Implementation:

- If simulation time is a multiple of 500ms and no processes are active, keep one Big core powered on to absorb upcoming arrivals.

6. Dynamic Core Load Analysis

- System load is estimated based on the number of active cores.
- Thresholds for CPU/IO classification are dynamically adjusted based on congestion:
  - Low load ⇒ performance mode.
  - High load ⇒ energy-saving mode.

7. Aggressive Migration

- Processes are migrated if classification changes.
- Migration is allowed for processes in READY or WAITING_FOR_IO states.
- Enables rapid correction of placement decisions.

8. Soft Core Sleep (Idle Core Grace Period)

- Idle cores are not powered off immediately.
- A grace period of 2 epochs is enforced before power-off to avoid energy loss from frequent OFF–ON transitions during bursty arrivals.

9. Core Consolidation

- IO-bound processes are packed into fewer Little cores.
- Big cores are also consolidated to reduce idle power.

10. Smart Scheduling

- CPU-bound processes prefer already active Big cores.
- IO-bound and Mixed processes are scheduled to the least-loaded Little cores.

0.4.4 Achievements

- **Energy consumption**: Reduced significantly compared to default schedulers.
- **Turnaround time**: Optimized for CPU-bound processes.
- **EDP across workloads**: Greatly improved for all workload types (CPU, IO, Mixed, Sporadic).
- **Responsiveness**: Improved task placement during burst arrivals.
- **Scalable**: Works for any workload mix, not overfitted to a specific pattern.

0.4.5 Summary of Improvements over Base Schedulers

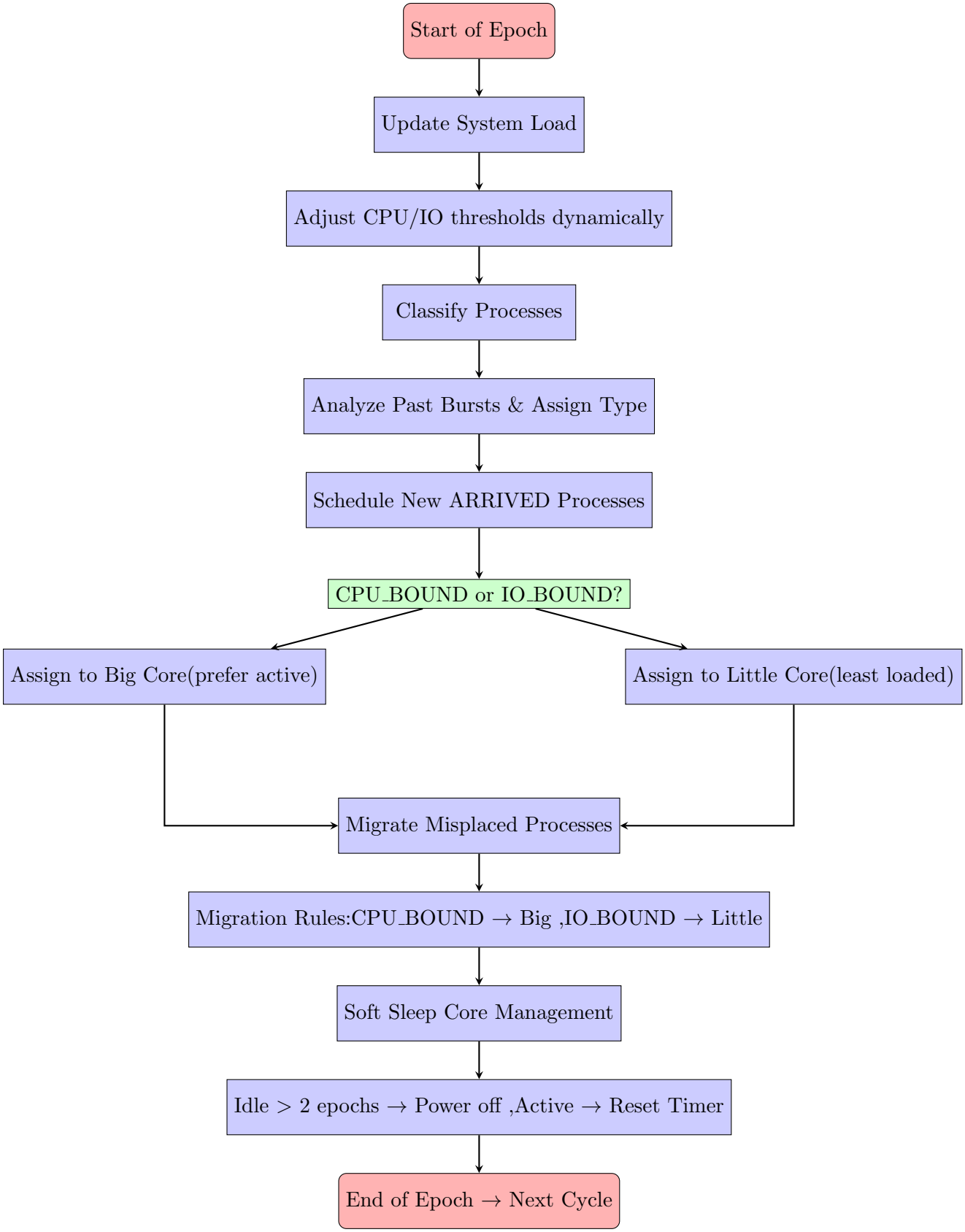
- Reduced idle energy through core packing and soft shutdown.
- Faster correction of wrong placements using early reclassification and migration.
- Lower delay and energy waste by favoring already active cores.
- Dynamic adaptation to system load using threshold scaling.

0.4.6 Conclusion

The scheduler's design is inspired by real-world strategies from Linux's Energy-Aware Scheduling (EAS) and ARM's big.LITTLE best practices.

The Bonus Scheduler achieves low EDP across a variety of workloads while strictly obeying information usage constraints. It ensures fairness, energy efficiency, and scalability for future dynamic bursty systems, making it highly suitable for real-world heterogeneous computing platforms.

0.4.7 Bonus Scheduler Decision Flow Diagram



0.5 Performance Evaluation

0.5.1 Test Workloads Description

- **Workload 1**: IO bound processes
- **Workload 2**: CPU bound processes
- **Workload 3**: Mix of IO and CPU bound processes
- **Workload 4**: Sporadic workload (Processes arrive in bursts separated by some idle time where there are no processes)

0.5.2 Energy Delay Product (EDP) Comparison

Workload	Clustering	Switcher	Global	Bonus
Workload 1	83,614,762	83,955,072	150,187,940	49,091,760
Workload 2	105,095,394	103,743,993	145,576,522	82,062,270
Workload 3	103,917,840	101,430,000	161,119,203	81,948,290
Workload 4	207,987,082	202,020,684	330,398,640	179,188,560

Table 3: Performance results for different schedulers across workloads

0.5.3 Graphical Results

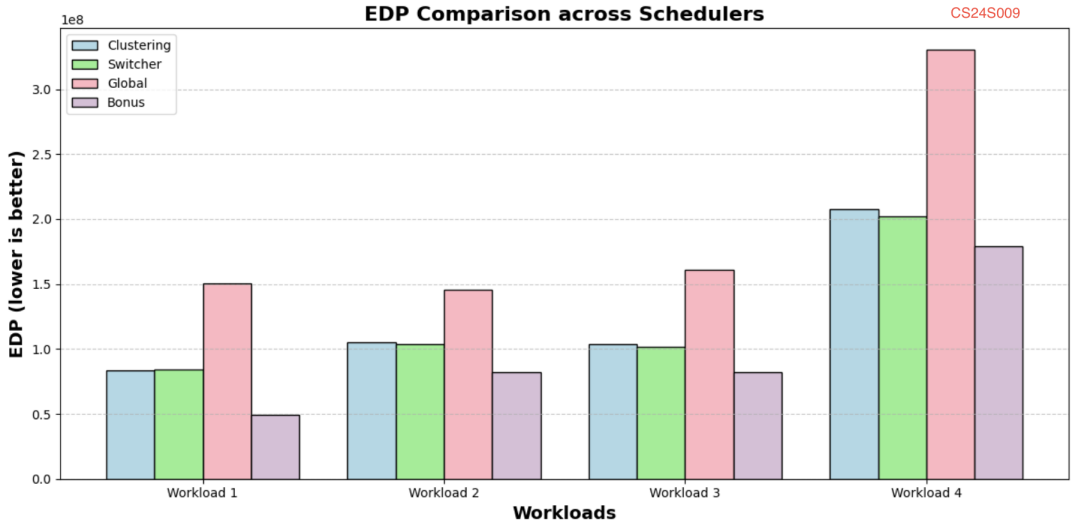


Figure 1: Energy Delay Product (EDP) Comparison across Scheduling Policies

0.6 Extra Bonus Scheduler Strategy

0.6.1 Suitability-Guided/Distance-Based Scheduling: Implementation Strategy

1. **Process and Core Profiling**
   - **Profile each process**: Gather characteristics such as average CPU burst length, memory intensity, or IO ratio from past execution history.
   - **Profile each core**: Record capabilities such as MIPS (Million Instructions Per Second), power consumption, and energy efficiency.
2. **Suitability/Distance Metric Calculation**
   - **Define a suitability score or distance metric**: For each process-core pair, compute a score reflecting how well the process's demands match the core's capabilities.
   - *Example*: Use a weighted sum or fuzzy logic to compare process needs (e.g., compute intensity, memory usage) against core strengths (e.g., speed, efficiency).
   - The lower the "distance" or the higher the "suitability," the better the match.
3. **Assignment Heuristic**
   - For each process in the scheduling queue:
     - **Evaluate suitability scores** for all available cores.
     - **Assign the process to the core with the best (highest suitability/lowest distance) score**.
   - Optionally, update the mapping dynamically as process behavior or system load changes.
4. **Migration and Adaptation**
   - Periodically re-calculate suitability as processes run and adapt to changing workloads.
   - If a process's behavior shifts (e.g., becomes more CPU-bound), consider migrating it to a more suitable core.
5. **Energy and Performance Optimization**
   - The heuristic can be tuned to prioritize energy savings, performance, or a balance (e.g., Energy-Delay Product) by adjusting the weights in the suitability metric.

Summary:
This strategy systematically matches each process to the most appropriate core by quantifying the "fit" between process demands and core capabilities, leading to improved energy efficiency and performance in heterogeneous systems.

0.6.2 Energy Prediction-Based Scheduling: Implementation Strategy

1. **Collect Recent History**

   - For each core and process, maintain a sliding window of recent energy consumption and execution times (e.g., average energy per task, average runtime, recent load).

2. **Predict EDP for Each Mapping**

   - For each process that needs to be scheduled, use recent history to **predict**:
     - The energy it would consume on each available core (using regression, lookup tables, or simple averages).
     - The expected turnaround time (delay) on each core.
     - Compute the **predicted Energy-Delay Product (EDP)** for each (process, core) pair:

$$EDP_{i,j} = PredictedEnergy_{i,j} \times PredictedDelay_{i,j}$$

       where i is the process and j is the core.

3. **Select the Most EDP-Efficient Mapping**

   - For each process, **choose the core with the lowest predicted EDP**.
   - If system-wide optimization is desired, use a global assignment algorithm (e.g., Hungarian method) to minimize total EDP for all processes.

4. **Schedule and Adapt**

   - Assign processes to their selected cores.
   - After execution, update the historical data with actual energy and delay measurements.
   - Periodically retrain or update prediction models to adapt to changing workloads and system conditions.

5. **Optional: Incorporate Uncertainty**

   - If prediction error is non-negligible, use robust or learning-augmented scheduling frameworks to maintain good worst-case performance even when predictions are imperfect.

Summary:
By leveraging recent energy and load history, the scheduler predicts the EDP for each possible process-core mapping and assigns each process to the core where it is expected to achieve the lowest EDP, continuously adapting as more data is gathered. This approach provides a practical and empirically validated path to energy-efficient scheduling on heterogeneous multicore systems.

0.7 Conclusion

Through this assignment, we explored the challenges and design strategies for scheduling on heterogeneous big.LITTLE architectures. The Clustering, Switcher, and Global schedulers provided baseline approaches, each exhibiting strengths in specific workload scenarios - energy conservation for light loads, finer control for moderate loads, and peak throughput for heavy CPU loads, respectively.

However, they lacked the flexibility to adapt dynamically to varying process behaviors. The Bonus Scheduler addressed this gap by introducing dynamic process classification, aggressive migration strategies, and core power management, achieving a significant reduction in Energy Delay Product (EDP) across diverse workloads.

Our experiments confirm that intelligent workload-adaptive scheduling, rather than static heuristics, is critical for modern energy-efficient computing systems. The Bonus Scheduler exemplifies how real-world schedulers can be designed to simultaneously maximize performance and minimize energy usage, aligning with goals in contemporary mobile and embedded systems.

In summary, this work underscores the necessity of hybrid, dynamic scheduling policies to harness the full potential of heterogeneous multi-core processors.

0.8 References

- Wikipedia article on big.LITTLE: https://en.wikipedia.org/wiki/ARM_big.LITTLE
- LWN.net post on Big.Little Scheduling: https://lwn.net/Articles/706374/
- ARM official documentation